Machine Learning Fundamentals

Multimedia Retrieval · Appendix A

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

A - Machine Learning Fundamentals 1
Tasks and Experience 2
Tasks 2
Experience 4
Losses and Performance Measures 7
Regression and the Mean Squared Error 7
Classification: Hard and Soft Assignments 8
Softmax 9
Cross-Entropy 9
Measures Without Ground Truth 10
Rewards in Reinforcement Learning 10
Over- and Underfitting 12
Capacity 12
Bias and Variance 12
Choosing the Capacity 14
Training and Model Selection 15
Training, Validation, and Test Sets 15
k-Fold Cross-Validation 15
The Training Pipeline 16
Summary 18
Key Takeaways 18
Key Formulas 18
Further Reading 18

A - Machine Learning Fundamentals

Many chapters of this book use a learned model somewhere: a classifier routes queries, an encoder turns text into embeddings, a network describes an image. This chapter collects the vocabulary these models share. It answers two questions: what does it mean for a program to learn, and how do we know that what it learned holds for data it has not seen? The methods themselves follow in the next chapters of this appendix.

Mitchell gave the definition that frames this chapter in a 1997 textbook:

> A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

The **task** is what we want the system to do, from a simple classification to driving a car. The **performance measure** says how well the system does the task; it guides training and lets us compare two systems. The **experience** is the data the system sees during training and what it is told about that data: examples with correct answers, rewards, or no feedback at all.

Where this is used

- The classifiers of Intent Routing and Classification, \#image-features-cnn, and Classifying Audio learn from labeled examples; Evaluating Text Classifiers and Scores, Thresholds, and ROC Curves evaluate them once they make hard decisions.
- The softmax and the cross-entropy loss train the word embeddings of Word Embeddings, and with a temperature they form the contrastive loss of Multimodal Transformers.
- Self-supervised learning is how the models of Transformer Embeddings and Automated Feature Extraction learn without manual labels.
- When Evaluation Misleads shows what happens when information from the test set leaks into training.
- C - Neural Networks builds on the gradient of the cross-entropy loss.

Prerequisites

Derivatives and the chain rule, vectors and norms, and basic probability. No other appendix chapter is needed; the following chapters on methods and neural networks build on this one.

Tasks and Experience describes what a system can learn to do and what it learns from. Losses and Performance Measures defines the measures that training minimizes and that compare systems, from the mean squared error to the softmax and the cross-entropy loss. Over- and Underfitting turns to the central difficulty, that a model can perform well on its training data and still fail on new data. Training and Model Selection describes the training protocol that detects and avoids this.

Tasks and Experience

Where this is used

Classifiers detect the language of a query and route it in Intent Routing and Classification, and they turn images and audio into labels in Automated Feature Extraction. Self-supervised learning trains the word embeddings of Word Embeddings and the transformer encoders of Transformer Embeddings, and contrastive learning aligns images with their captions in Multimodal Transformers. The language models of 7 - Retrieval Augmented Generation and the speech recognizers of Transcription perform two of the tasks described below. \#image-features-cnn reuses learned representations for a new task with only six labels.

Mitchell's definition in A - Machine Learning Fundamentals names three ingredients of learning: a task T, a performance measure P, and experience E. This section covers the task and the experience: what we can ask a system to learn, and what it learns from. Losses and Performance Measures covers the performance measure.

Tasks

A task describes how the system should process an **example**, a set of features that we write as a vector $\boldsymbol{x} \in \mathbb{R}^M$: the term weights of a document, the pixel values of an image, or the MFCCs of an audio frame. Learning is not the task itself but the means to perform it. The list below follows the catalogue in the deep learning textbook by Goodfellow, Bengio, and Courville (see Further Reading) and adds the tasks that appear in this book. Many more exist.

Classification maps an example to one of K categories: the system learns a function $f : \mathbb{R}^M \to \{1, \ldots, K\}$. A common variant outputs a probability distribution $P(y)$ over the K classes instead of a single class, with probabilities that sum to 1. Applications include object recognition in images, text categorization, spam filtering, handwriting and speech recognition, and credit scoring. In this book, classifiers detect the language of a query and route it to a backend (Intent Routing and Classification), and they turn images and audio into labels (Automated Feature Extraction).

Classification with missing inputs allows some features of an example to be absent. In disease prediction, the target "healthy" or "has the disease" depends on laboratory results, and since tests are expensive, not every patient has every result. A system that needs one function per subset of available features would need 2^M functions for M features. A better approach learns a probability distribution over all features and marginalizes out the missing ones, that is, it averages over all values they could take, weighted by their distribution in the training data. One model then serves every subset. Naive Bayes (Naive Bayes) handles missing features naturally: it chooses the class $k^* = \operatorname{argmax}_k P(C_k) \prod_i P(x_i \mid C_k)$, and the product simply runs over the observed features only. Every task below has a similar variant with missing inputs.

Regression predicts a number: the system learns a function $f : \mathbb{R}^M \to \mathbb{R}$. Unlike classification, the output is a real value, not a choice among categories and not a distribution over all possible values. Applications include forecasts, algorithmic trading, expected claims in insurance, financial risk assessment, budgeting, and pricing together with its effect on sales. Figure 0.1 contrasts the two tasks: in classification, the target is a label, drawn as a shape; in regression, it is a value on the vertical axis.

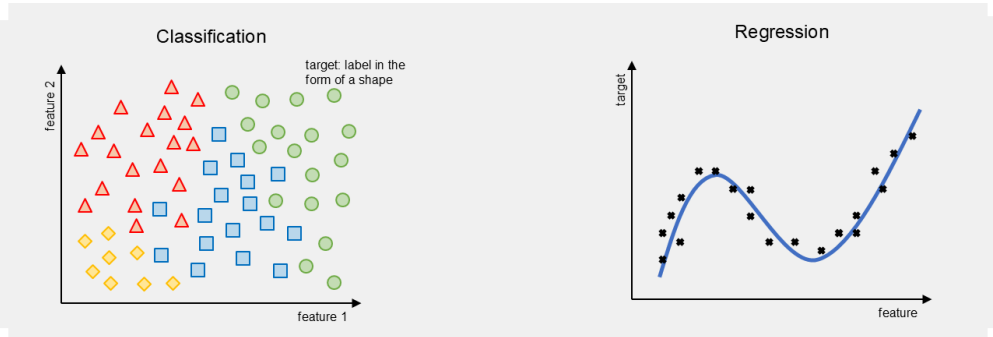


Figure 0.1: Training examples for classification (left), with labels shown as shapes, and for regression (right), with numeric targets.

Clustering divides a set of examples into groups. Unlike classification, the groups are not known in advance, and the algorithm must discover them. Because there are no expected outputs, clustering is an *unsupervised* task, while classification and regression are *supervised* (see Experience below). Applications include genetic clustering, market segmentation into customer groups, communities in social networks, image segmentation, anomaly detection, and crime analysis. Clustering describes k-means and expectation maximization; Approximate Nearest-Neighbor Search uses k-means to compress vectors for search.

Density estimation constructs an estimate of an unknown probability distribution from examples. The system learns a function $p : \mathbb{R}^M \to \mathbb{R}$, where $p(\boldsymbol{x})$ is a probability density for continuous $\boldsymbol{x}$ or a probability mass function for discrete $\boldsymbol{x}$. The simplest estimator is a histogram. Figure 0.2 estimates the same distribution with wide and with narrow bins. Wide bins smooth away detail, and narrow bins show spikes that come from the particular sample rather than from the distribution. The bin width is a first example of the capacity of a model, which returns in Over- and Underfitting. Applications include the age distribution of a country, feature extraction, and the simplification of models. Density estimation is also a building block of other tasks: anomaly detection flags examples with a low density, and classification with missing inputs marginalizes over a learned density.

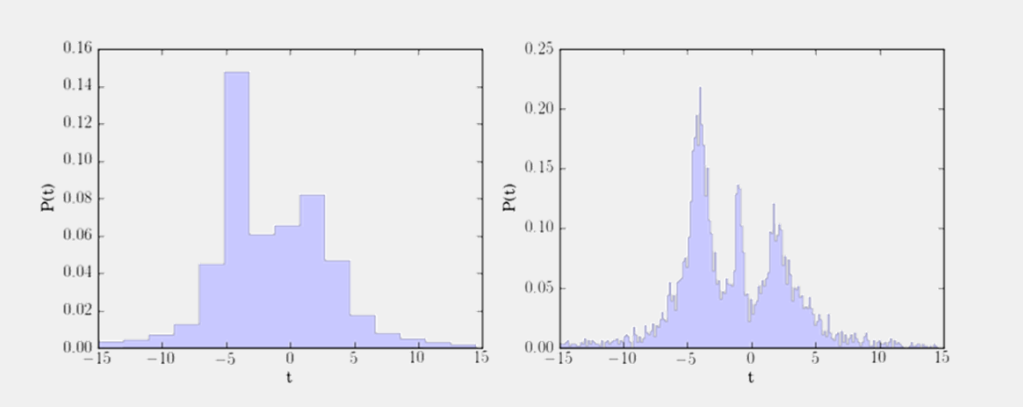


Figure 0.2: Histogram estimates of the same distribution $P(t)$ with wide bins (left) and narrow bins (right).

Imputation of missing values replaces missing data with estimated values. Given an example $\boldsymbol{x} \in \mathbb{R}^M$ with some entries x_i missing, the system predicts the missing entries. Applications include incomplete sensor data, incomplete personal records in demographics, medical analysis with missing or expensive tests, and signal restoration after data loss.

Anomaly detection identifies unusual, incorrect, or atypical events. The output is a flag, 1 for an anomaly and 0 otherwise, or a probability of an anomaly. Supervised anomaly detection learns from examples labeled "normal" and "abnormal". Unsupervised anomaly detection describes the normal behavior, for example with a density estimate, and flags the examples that deviate from it. Applications include credit card fraud detection, intrusion detection, removing outliers before a statistical analysis, change detection, system health monitoring, and fault detection.

Generation produces new examples that resemble the training data: images, text, music, or program code. The model captures the patterns of the training data and samples new instances with similar characteristics. Generative models usually learn without manual labels, through self-supervised learning (see Experience below). Applications include image synthesis, text generation, style transfer, data augmentation, chatbots, code assistants, assets for games, and summarization. The language models that write the answers in 7 - Retrieval Augmented Generation belong here.

Machine translation maps a sequence of symbols in one language to a sequence in another. Translating word by word is not enough; the system must find a representation that is structurally and semantically correct in the target language.

Transcription converts unstructured data into a discrete, usually textual form. Optical character recognition and speech recognition (Transcription) are the best-known examples.

Dimensionality reduction maps examples to a space with fewer dimensions. The new dimensions can often be read as topics, concepts, or embeddings, which makes it easier to find documents on similar topics. Principal component analysis and latent semantic indexing (Latent Semantic Indexing) are classic methods; the embeddings of Word Embeddings are learned ones. Other uses include data mining, statistical analysis, and compression.

Reasoning derives conclusions from knowledge with logical techniques such as deduction and induction. Knowledge-based systems, for example expert systems written in Prolog, use facts and rules to prove or disprove statements within a closed world. Machine learning now supports theorem proving and constraint solving, and large language models produce intermediate reasoning steps before an answer, which improves their results on problems that need several steps.

Autonomous agents, such as robots and self-driving cars, act in an environment and adjust their behavior to rewards and penalties, which is reinforcement learning (see Experience below). They must decide in situations that their programmers could not anticipate or hard-code. This raises questions of machine ethics: when a collision with an animal or a person is unavoidable, should the car risk an evasive maneuver that endangers its passengers? Liability for mistakes, safety, privacy, and the hacking of vehicles remain open questions for law and society.

Experience

The experience is the data that the system observes during training and what it is told about that data. The types of experience differ in the feedback they provide: correct outputs, outputs for some examples only, no outputs, rewards, or outputs taken from the data itself.

Supervised Learning

A supervised learning algorithm observes a dataset of features together with their targets. It learns a general rule that maps features to targets, so that it can predict the targets of new examples. The term comes from the idea of a teacher who provides the targets and so shows the algorithm how features map to labels. Figure 0.1 shows both kinds of targets. The goal is not to reproduce the targets of the training set but to generalize to new data. The teacher therefore usually provides a performance measure along with the labels, which checks how well the rule generalizes. Over- and Underfitting explains why the two goals differ.

Labels are expensive. Creating them is laborious work, much like producing metadata for a collection (Metadata and Its Limits). This motivates approaches that need fewer labels or none: semi-supervised and self-supervised learning.

Semi-Supervised Learning

In semi-supervised learning, only some training examples have targets. In credit card fraud detection, for example, only the transactions that were investigated carry the label “fraud” or “no fraud”, and the vast majority remains unlabeled. To learn from such data, algorithms make assumptions about how the labels relate to the distribution of the features:

- **Smoothness**: points close to each other share the same label, so the label changes smoothly across the feature space.
- **Cluster**: the data forms clusters, and all objects in a cluster share the same label.
- **Manifold**: the features are high-dimensional, but the data lies close to a low-dimensional manifold.

Figure 0.3 contrasts two ways to use the unlabeled points. **Induction** learns a general rule that labels any new point. Its simplest form trains a supervised model on the labeled examples and applies it to the unlabeled ones. This works when only a few labels are missing. When most labels are missing, the labeled part is too small to capture the distribution of the labels, and the approach ignores most of the data.

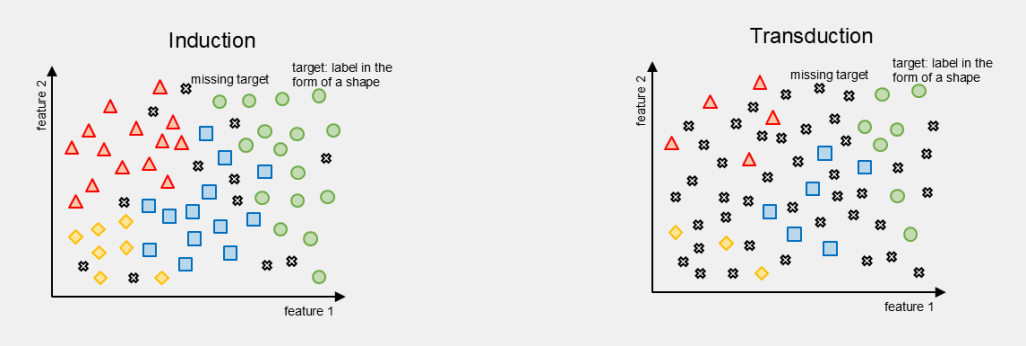


Figure 0.3: Semi-supervised learning with labels shown as shapes and missing targets as crosses: induction (left) and transduction (right).

Transduction has a narrower goal: it learns no rule for points that arrive later, but only predicts the labels of the unlabeled points already in the dataset. For this, it uses the positions of all points, labeled and unlabeled, because together they show where the clusters are. A simple partitioning method builds the clusters as follows. It starts with one cluster that contains all objects. While a cluster contains two labeled objects with different labels, it splits the cluster to resolve the conflict. Finally, it gives every unlabeled object the label of the labeled objects in its cluster. Many variants exist for building the clusters.

Unsupervised Learning

Unsupervised learning algorithms observe data without targets and derive a function that describes its structure or distribution. No teacher provides targets or evaluates the result; the algorithm must find meaningful patterns on its own. Figure 0.4 shows the two most common findings, a cluster and an outlier. Typical tasks are:

- **Clustering**: find groups of similar objects based on a distance function. The number of groups is often unknown.
- **Outlier and anomaly detection**: learn the normal behavior and identify the objects that deviate strongly from it. The training data may itself contain outliers.
- **Density estimation**: describe the data with a density function, from a single Gaussian to the best fit among several families of distributions.
- **Dimensionality reduction**: extract the core concepts of high-dimensional features, for example with principal component analysis, for a simpler yet accurate view of the data.
- **Self-organizing maps**: map high-dimensional data onto a two-dimensional grid by competitive learning.

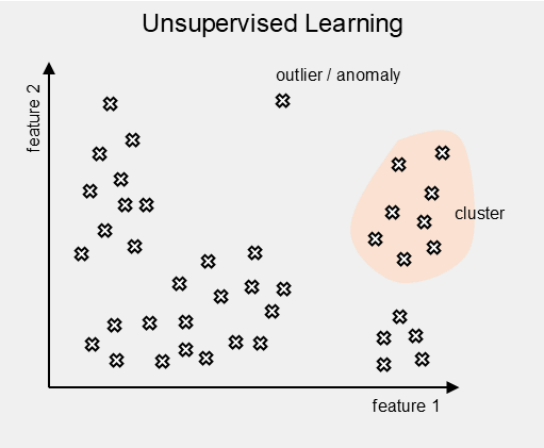


Figure 0.4: Unlabeled data with a cluster and an outlier.

Reinforcement Learning

Reinforcement learning receives neither targets nor data alone, but rewards for its actions; Losses and Performance Measures describes how the reward function serves as its performance measure. One prominent use connects it with language models. In reinforcement learning from human feedback, people rank

several answers of a model, a reward model learns these preferences, and reinforcement learning then tunes the language model to produce answers with a high reward (Ouyang et al., see Further Reading).

Self-Supervised Learning

Self-supervised learning takes the targets from the data itself. It hides part of an example and trains the model to predict the hidden part from the rest. No manual labels are needed, so the model can learn from collections with billions of examples. Technically, it is supervised learning with free labels. Several models in this book learn this way. Word2vec predicts the words around a center word (Word Embeddings). BERT masks 15% of the tokens of a sentence and predicts them from the context (Transformer Embeddings), and generative language models predict the next token. Contrastive learning pairs data from different modalities: CLIP learns that an image and its caption from the web belong together (Multimodal Transformers). Pairs collected from the web are noisy, so this setting is also called weak supervision.

Learned representations, whether they come from supervised or self-supervised training, serve new tasks with few labels. \#image-features-cnn trains a classifier on frozen image embeddings with only six labeled photos.

Generative adversarial networks, introduced by Goodfellow and colleagues in 2014, also create their own labels. Figure 0.5 shows the two networks. A generator turns random noise into a sample, for example an image. A discriminator receives real samples from the dataset and generated ones and predicts for each whether it is real or generated. The labels "real" and "generated" come for free. The two networks compete: the generator learns to make the discriminator wrong, and the discriminator learns to tell the two kinds of samples apart. Diffusion models have since replaced generative adversarial networks in most image generators.

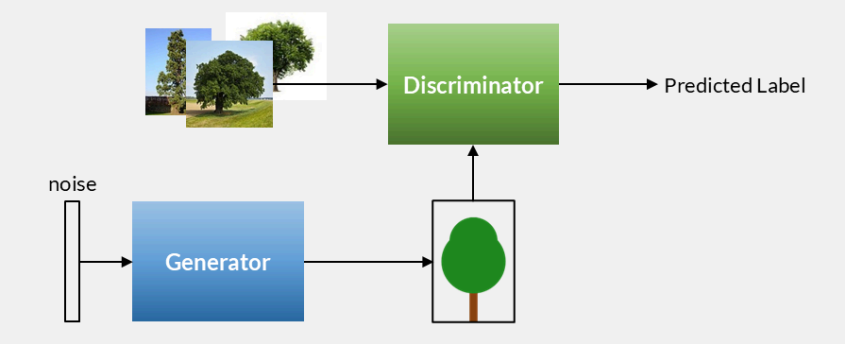


Figure 0.5: A generative adversarial network: the discriminator predicts whether an image is real or was produced by the generator from noise.

Losses and Performance Measures

Where this is used

Evaluating Text Classifiers and Scores, Thresholds, and ROC Curves measure classifiers once they make hard decisions; this section covers the soft assignments with which they are trained. The softmax and the cross-entropy loss train the word embeddings of Word Embeddings, and with a temperature they form the contrastive loss of Multimodal Transformers. \#image-features-cnn puts a softmax on top of frozen image embeddings. Approximate Nearest-Neighbor Search trades search quality against speed and memory, the trade-off by which dimensionality reduction is measured. C - Neural Networks builds on the gradient of the cross-entropy loss derived below.

A performance measure, the P in Mitchell's definition, states how well a system performs its task, and each task has its own idea of what "good" means. Measures serve two purposes. During training, a measure called the **loss** is what the learning algorithm minimizes. After training, a measure compares systems with each other. The two need not be the same: we usually train a classifier with the cross-entropy loss below and report its accuracy or F1 score. Measures fall into three groups: supervised learning compares the outputs with known targets, unsupervised learning has no targets, and reinforcement learning receives rewards.

Regression and the Mean Squared Error

A regression model is a function $f_{\boldsymbol{w}} : \mathbb{R}^M \to \mathbb{R}$ with parameters $\boldsymbol{w} \in \mathbb{R}^D$. The number of parameters D depends on the chosen function: a line has two, a neural network may have billions. For N examples $\boldsymbol{x}_i$ with targets y_i, the model predicts $\hat{y}_i = f_{\boldsymbol{w}}(\boldsymbol{x}_i)$. The standard measure is the mean squared error between the vector of predictions $\hat{\boldsymbol{y}}$ and the vector of targets $\boldsymbol{y}$.

Key Formula: Mean Squared Error

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 = \frac{1}{N}\parallel \hat{\boldsymbol{y}} - \boldsymbol{y} \parallel_2^2 \tag{0.1}$$

The average squared distance between prediction and target. Squaring makes all errors positive and lets large errors count far more than small ones.

Training searches for the parameters $\boldsymbol{w}^*$ with the smallest error. The factor $\frac{1}{N}$ does not change where the minimum lies, so we omit it:

$$\boldsymbol{w}^* = \text{argmin}_{\boldsymbol{w}} \sum_{i=1}^{N}(f_{\boldsymbol{w}}(\boldsymbol{x}_i) - y_i)^2 \tag{0.2}$$

At a minimum, the gradient with respect to the parameters is zero:

$$\nabla_{\boldsymbol{w}} \sum_{i=1}^{N}(f_{\boldsymbol{w}}(\boldsymbol{x}_i) - y_i)^2 = \boldsymbol{0} \tag{0.3}$$

For simple models such as a line, this condition can be solved exactly with calculus. For complex models, **gradient descent** finds a numerical solution. It starts with some parameters and repeatedly takes a small step against the gradient, $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\nabla_{\boldsymbol{w}}\,\text{MSE}$, where the step size η is called the learning rate. Gradient descent may stop in a local minimum, so its result can be approximate. The squared error keeps the gradient simple: the derivative of $(\hat{y}_i - y_i)^2$ is $2(\hat{y}_i - y_i)$ times the derivative of $\hat{y}_i$. Backpropagation in C - Neural Networks trains the weights of a network with the same idea, using stochastic gradient descent.

Example: Fitting a line to four points

Four examples have the features $x = 1, 2, 3, 4$ and the targets $y = 1, 3, 2, 5$. The model is a line $\hat{y} = ax + b$ with the parameters $\boldsymbol{w} = (a, b)$.

A first guess $a = 1, b = 0$ predicts $1, 2, 3, 4$. The errors $\hat{y}_i - y_i$ are $0, -1, 1, -1$, so MSE $= \frac{0+1+1+1}{4} = 0.75$.

The gradient tells us how to improve the guess:

$$\frac{\partial\,\mathrm{MSE}}{\partial a} = \frac{2}{N}\sum_i (\hat{y}_i - y_i)\,x_i = \frac{2}{4}(0 - 2 + 3 - 4) = -1.5, \qquad \frac{\partial\,\mathrm{MSE}}{\partial b} = \frac{2}{N}\sum_i (\hat{y}_i - y_i) = \frac{2}{4}(0 - 1 + 1 - 1) = -0.5$$

Both derivatives are negative, so increasing a and b lowers the error. One gradient step with $\eta = 0.1$ gives $a = 1.15$ and $b = 0.05$, and the error drops to MSE $= 0.709$.

For a line, we can also solve the condition "gradient equals zero" directly. Setting both derivatives to zero gives two linear equations, the normal equations. With $\sum x_i = 10$, $\sum x_i^2 = 30$, $\sum y_i = 11$, and $\sum x_i y_i = 33$, they read:

$$30a + 10b = 33, \qquad 10a + 4b = 11 \tag{0.5}$$

The solution is $a = 1.1$ and $b = 0$. The line predicts $1.1, 2.2, 3.3, 4.4$, the errors are $0.1, -0.8, 1.3, -0.6$, and MSE $= \frac{0.01+0.64+1.69+0.36}{4} = 0.675$. No other line has a smaller error on these four points.

Classification: Hard and Soft Assignments

Classification tasks differ along two dimensions. The first is the number of classes: **binary** classification decides between two classes, **multi-class** classification between many, often hundreds. Once a classifier makes hard decisions, precision, recall, accuracy, and the other values of the confusion matrix measure its quality. Evaluating Text Classifiers covers them for two and for many classes, and Scores, Thresholds, and ROC Curves shows how a score becomes a decision. This subsection is about the second dimension, which matters for training.

The second dimension is the form of the output. A **hard assignment** names one class. A **soft assignment** gives a probability for each class. Figure 0.6 shows both for an image classifier with four classes: dogs, cats, horses, and sharks. The network does not output probabilities directly. It outputs four raw scores, the **logits**, here 1.3, 4.6, 0.4, and 2.1. The soft assignment turns them into probabilities: 3% for dogs, 88% for cats, 1% for horses, and 7% for sharks. The photo shows a cat in a knitted shark costume, which explains the 7% for sharks. The hard assignment keeps only the most likely class, sets its output to 1 and all others to 0, and says "it is a cat".

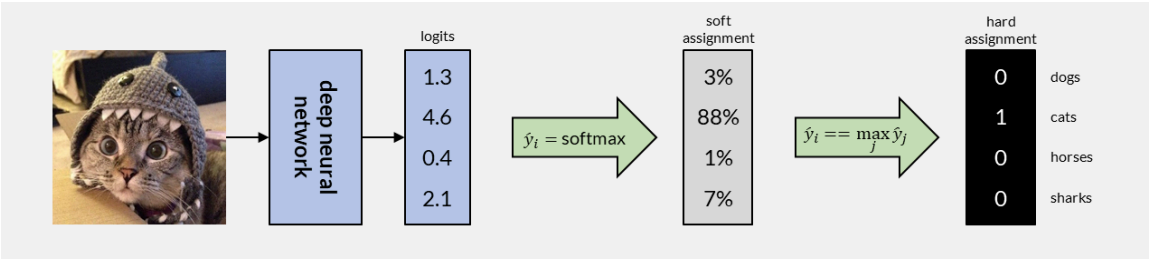


Figure 0.6: The logits of an image classifier, turned into a soft assignment by the softmax and into a hard assignment by keeping the largest value.

Soft assignments carry information that hard assignments lose. A second system that gives the cat only 79% makes the same hard assignment, so accuracy rates both systems equally, yet the first is more confident in the correct class. For training, there is a second reason to prefer soft assignments. Accuracy changes only when a prediction flips from one class to another, so a small change of the parameters usually leaves it unchanged, and its gradient is zero almost everywhere. Gradient descent needs a measure that changes smoothly with the parameters. This requires two steps: a function that maps logits to probabilities, and a loss that measures how well these probabilities match the 0/1 targets of the training set.

Softmax

The logits $o_k \in \mathbb{R}$, one for each of the K classes, can take any value, and their range depends on the model and its activation functions. We need probabilities $\hat{y}_k$ that are positive, sum to 1, and keep the order of the logits. Many functions satisfy these conditions; the **softmax** is the standard choice.

Key Formula: Softmax

$$\hat{y}_k = \frac{\exp(o_k)}{\sum_{j=1}^{K} \exp(o_j)} \tag{0.6}$$

The exponential makes every value positive and preserves the order of the logits; dividing by the sum makes the values add up to 1. A difference of one between two logits becomes a factor of $e \approx 2.7$ between their probabilities.

Unless stated otherwise, exp and log use the natural base e.

Example: Softmax of the four logits

For the logits $(1.3, 4.6, 0.4, 2.1)$ of Figure 0.6, the exponentials are 3.669, 99.484, 1.492, and 8.166, with the sum 112.812. Dividing each by the sum gives

$$\hat{\boldsymbol{y}} = (0.033, 0.882, 0.013, 0.072) \tag{0.7}$$

which are the 3%, 88%, 1%, and 7% of the figure. Adding the same constant to every logit does not change the result, because the factor $\exp(c)$ cancels between numerator and denominator. Implementations use this to avoid overflow and subtract the largest logit first: the shifted logits $(-3.3, 0, -4.2, -2.5)$ give the same probabilities.

Dividing all logits by a **temperature** τ before the softmax controls how sharp the distribution is. A temperature below 1 sharpens it toward the largest logit, and a temperature above 1 flattens it. For the four logits above, $\tau = 0.5$ gives the cat 99% and $\tau = 2$ only 62%. The contrastive loss of Multimodal Transformers uses a small temperature.

Cross-Entropy

In information theory, the cross-entropy measures how well a model distribution q matches a true distribution p over a set of events, $H(p, q) = -\sum_e p(e) \log q(e)$. It is smallest when $q = p$; Information Theory shows why it is the standard loss. In classification, the true distribution of a training example is a **one-hot** vector $\boldsymbol{y}$: the component of the correct class is 1, and all others are 0. The model distribution is the softmax output $\hat{\boldsymbol{y}}$.

Key Formula: Cross-Entropy Loss and Its Gradient

$$J(\boldsymbol{y}, \hat{\boldsymbol{y}}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k, \qquad \frac{\partial J}{\partial o_k} = \hat{y}_k - y_k \tag{0.8}$$

With a one-hot target, the loss is the negative log probability of the correct class: zero when the model gives the correct class the probability 1, and growing without bound as that probability approaches 0. Its gradient with respect to each logit is the prediction minus the target.

The gradient follows in three steps. Inserting the softmax gives $\log \hat{y}_k = o_k - \log \sum_j \exp(o_j)$, so

$$J = -\sum_k y_k \, o_k + \sum_k y_k \log \sum_j \exp(o_j) = \log \sum_j \exp(o_j) - \sum_k y_k \, o_k \tag{0.9}$$

where the second step uses $\sum_k y_k = 1$. The derivative of the second term with respect to o_k is y_k. The first term needs the chain rule: if $F(x) = f(g(x))$, then $F'(x) = f'(g(x)) \cdot g'(x)$. With $f = \log$ and $g = \sum_j \exp(o_j)$, we get

$$\frac{\partial}{\partial o_k} \log \sum_j \exp(o_j) = \frac{\exp(o_k)}{\sum_j \exp(o_j)} = \hat{y}_k \tag{0.10}$$

Together, $\frac{\partial J}{\partial o_k} = \hat{y}_k - y_k$. The base of the logarithm does not matter for training: another base multiplies the loss by a constant and leaves the minimum where it is.

Example: Loss and gradient for the cat photo

The correct class of the photo is cats, so $\boldsymbol{y} = (0, 1, 0, 0)$. With $\hat{y}_{\text{cat}} = 0.882$ from the softmax example, the loss is $J = -\log 0.882 = 0.126$. The second system with 79% for cats has the loss $-\log 0.79 = 0.236$, almost twice as large, although both make the same hard assignment.

The gradient of the first system is

$$\hat{\boldsymbol{y}} - \boldsymbol{y} = (0.033, -0.118, 0.013, 0.072) \tag{0.11}$$

Gradient descent moves against the gradient: it raises the logit for cats and lowers the other three, the logit for sharks the most, because sharks received most of the probability that cats lacked. The components sum to zero, since both $\hat{\boldsymbol{y}}$ and $\boldsymbol{y}$ sum to 1.

The softmax belongs in the loss, not twice in the model

Deep learning libraries often combine the softmax and the cross-entropy into one function for numerical stability. `torch.nn.CrossEntropyLoss` in PyTorch, for example, expects logits, not probabilities. A model that already ends with a softmax layer then applies the softmax twice. Training still runs, but the inputs to the second softmax lie between 0 and 1, so the correct class can never receive more than $e/(e + K - 1)$, which is 48% for four classes, and the loss cannot fall below 0.74.

Measures Without Ground Truth

Unsupervised tasks have no targets, so the measures above do not apply. Instead, we judge whether the model found "good" clusters or "true" outliers.

For clustering, the central difficulty is the number of clusters and their shape. Too many clusters split coherent regions; too few merge groups that should stay apart. Internal measures need no labels: the elbow method, the silhouette score, and the Davies-Bouldin index rate how compact the clusters are and how well they are separated. Clustering describes the first two. When a reference partition exists, for example when we test a clustering algorithm on labeled data, external measures such as the adjusted Rand index compare the clusters with it.

Dimensionality reduction methods such as principal component analysis or autoencoders compress the data. Their measure is how well the original data can be reconstructed from the reduced representation. Often, the reduced vectors serve another task, such as vector search, and the measure becomes a trade-off: faster search and less memory against the loss of quality compared with search on the uncompressed vectors (Approximate Nearest-Neighbor Search).

Rewards in Reinforcement Learning

A reinforcement learning agent acts in an environment in discrete time steps. In each step, it observes the state, chooses an action, and receives a reward. Its goal is to maximize the cumulative reward. There are no known pairs of inputs and correct outputs, so the agent must balance the exploration of unknown situations against the exploitation of what it already knows. The field draws on game theory, control theory, simulation, and genetic algorithms. The reward function is the performance measure, and designing it well is often the hardest part:

- **Autonomous driving**: positive rewards for following traffic rules, staying on the road, and avoiding collisions; negative rewards for breaking rules, driving too fast or erratically, and causing accidents.
- **Game playing**: rewards for the strategic advantages that lead to a win. In chess, the reward can be the final win or loss alone, or it can add intermediate rewards for capturing pieces and controlling

the center. Intermediate rewards speed up learning but can teach the agent to value material more than the win.

- **Financial trading**: a reward that balances profit against risk. An agent that chases highly profitable trades leaves the risk profile of the portfolio owner, while an overly cautious one misses opportunities. The strategy must also withstand unexpected changes of the market.

Over- and Underfitting

Where this is used

When Evaluation Misleads describes how a retrieval system can overfit to a test collection that stays fixed for years. Automated Feature Extraction names dropout and data augmentation as two of the methods that kept deep networks from overfitting, and C - Neural Networks describes them. \#image-features-cnn trains a linear classifier on only six labeled photos, a setting in which the capacity of the model must stay small.

A model that performs well on its training data is not necessarily a good model. Our goal is a model that performs well on new data it has not seen, which is called **generalization**. A model that memorizes the training data performs perfectly on it and poorly on everything else.

Capacity

Capacity describes how many different functions a model can represent. A line $y = ax + b$ has two parameters and can only represent straight lines. A polynomial of degree d has $d + 1$ parameters and can change direction up to $d - 1$ times. A model with too little capacity cannot capture the pattern in the data, and one with too much capacity also captures the noise. Two rules that predict whether it is warm show the extremes. A model that is too simple says: "if the sun is out, it is warm". A model that is too complex says: "if the sun is out, and it is a summer month and we are in the northern hemisphere or a winter month and we are in the southern hemisphere, or we are near the equator, or in a desert that is not an ice desert, and it is not cloudy, raining, or snowing, and there is no strong wind, no solar eclipse, and no volcanic eruption, and we are not in the water, in a cave, in the shade, in an air-conditioned house or car, or in a freezer, then it is warm".

Humans are good at finding the right level of abstraction. "Birds can fly" is a simple model, and we learn the exceptions, such as penguins, separately. The question of what makes a chair a chair has no simple answer in terms of shape, so we use an abstract concept: we can sit on it. Horses come in a narrow variety of forms, and we draw clear limits on what counts as a horse; donkeys, zebras, and giraffes do not. Dogs come in a wide variety of forms, yet we recognize them immediately.

Figure 0.7 shows the two failures for classification and regression. **Underfitting** occurs when the model is too simple for the pattern in the data, for example a straight line for a curved relationship. Its error is large already on the training data, and it predicts new data poorly. **Overfitting** occurs when the model follows the training data too closely, typically because it has too many parameters relative to the number of training examples. Its training error is small, but it fails on new data, because it has learned the noise of the particular sample. The optimal model has small errors on both the training data and new data.



Figure 0.7: Underfitting, optimal fit, and overfitting for a classification boundary (left) and a regression curve (right).

Bias and Variance

The two failures correspond to two kinds of error. **Bias** is the error of a model that is too simple to represent the true relationship; a model with high bias misses essential patterns no matter how much data it sees.

Variance is the sensitivity of a model to the particular training sample; a model with high variance changes a lot when a few training examples change, and it performs well on its training data but poorly on new data.

For the squared error, the expected error at a point x splits exactly into three terms. Here $f(x)$ is the true value, $y = f(x) + \varepsilon$ is an observation with noise ε of variance σ^2, and $\hat{f}(x)$ is the prediction of a model trained on a random training set. The expectation runs over the training sets and the noise:

$$\mathbb{E}\Big[\big(\hat{f}(x) - y\big)^2\Big] = \Big(\mathbb{E}\big[\hat{f}(x)\big] - f(x)\Big)^2 + \mathbb{E}\Big[\big(\hat{f}(x) - \mathbb{E}\big[\hat{f}(x)\big]\big)^2\Big] + \sigma^2 \tag{0.12}$$

The first term is the squared bias: how far the average model is from the truth. The second is the variance: how much the individual models scatter around the average model. The third is the noise in the data, which no model can remove. Reducing the bias by adding capacity usually increases the variance, and the reverse. The goal is therefore the model with the smallest sum of squared bias and variance.

Example: Polynomials of increasing degree

We draw ten training points at $x = 0, \frac{1}{9}, \frac{2}{9}, \ldots, 1$ with the targets $y = \sin(2\pi x) + \varepsilon$, where the noise ε has a standard deviation of $\sigma = 0.2$, so $\sigma^2 = 0.04$. We fit polynomials of degree d by minimizing the mean squared error. To measure bias and variance, we repeat the experiment 2,000 times: each repetition keeps the ten positions x and draws new noise, which gives a new training set from the same distribution. Averaged over x in $[0, 1]$ and over the training sets, we obtain the following errors. The expected test MSE is the sum of the squared bias, the variance, and the noise σ^2.

Degree d	Parameters	Training MSE	Bias²	Variance	Expected test MSE
0	1	0.489	0.500	0.004	0.544
1	2	0.297	0.222	0.007	0.270
3	4	0.030	0.007	0.013	0.060
5	6	0.016	< 0.001	0.021	0.061
7	8	0.008	< 0.001	0.036	0.076
9	10	0.000	< 0.001	0.266	0.306

The training error falls with every degree and reaches zero at $d = 9$, where ten parameters let the polynomial pass through all ten points. The test error falls only up to $d = 3$ and then rises again. Degrees 0 and 1 underfit: their error is almost all bias. Degree 9 overfits: its bias is negligible, but its variance is so large that it predicts new data worse than a straight line. Degree 3 comes closest to the noise floor of 0.04, and degree 5 is nearly as good.

In practice, we cannot compute the bias and the variance, because we do not know f and have only one training set. Instead, we compare the loss on the training data with the loss on data held out from training, the validation set of Training and Model Selection. Underfitting shows as a high loss on both. Overfitting shows as a small training loss and a much higher validation loss. The validation loss is usually somewhat higher than the training loss, and a large gap is the warning sign. Figure 0.8 shows the typical shape as the capacity grows. The training loss keeps falling, while the generalization error falls, reaches a minimum at the optimal capacity, and rises again. The distance between the two curves is the **generalization gap**.

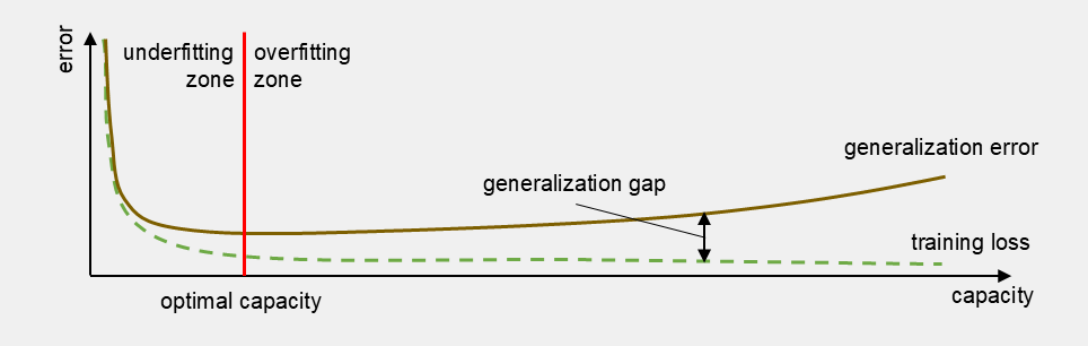


Figure 0.8: Training loss and generalization error as functions of the model capacity, with the underfitting and overfitting zones on either side of the optimal capacity.

Choosing the Capacity

An old heuristic for finding the right capacity is **Occam's razor**, named after William of Ockham (c. 1287-1347): *Numquam ponenda est pluralitas sine necessitate*, "plurality must never be posited without necessity". In its modern reading, it says that when several hypotheses explain the observations equally well, we should prefer the simplest one. Statistical learning refined the idea in the 20th century. For building models, it means starting with low capacity and adding capacity only as far as the task requires. Physics offers an example: Newton's laws simplify the real world, yet they are accurate enough in many situations.

Simpler models generalize better, but a model must not be so simple that its training loss stays high. Modern language models have billions of parameters, enough to memorize large parts of their training data. **Regularization** keeps them from doing so by penalizing complex models; C - Neural Networks describes the methods. L2 regularization, for example, adds a penalty for large weights to the loss, so the network prefers many small weights over a few large ones, and L1 regularization drives many weights to exactly zero.

Very large models also behave differently from the curve in Figure 0.8. Beyond the capacity at which the training error reaches zero, the test error can fall again, an effect known as double descent (Belkin et al., see Further Reading). The polynomial example stops exactly at this threshold, with ten parameters for ten points.

Training and Model Selection

Where this is used

Scores, Thresholds, and ROC Curves chooses a decision threshold on validation data, and When Evaluation Misleads shows what goes wrong when the test set shapes the model. Classifying Audio holds out one whole recording at a time and trains on the others, a leave-one-out cross-validation over recordings instead of over segments.

Training is one step of a larger lifecycle. MLOps combines it with the collection and curation of data, the integration of the model into an application, and the monitoring of the model in operation, all in one continuous cycle (Gupta, see Further Reading). This section describes the core of that cycle: how to train a model and choose among models so that the result generalizes.

Training, Validation, and Test Sets

The first step sets data aside for the final assessment. We split the data into a **training set** and a **test set**, typically 80% and 20%, and do not use or even look at the test set until the very end. Its only purpose is to estimate how the final model will perform on new data. Machine learning competitions go further: participants train with the training data only, and the organizers evaluate the submitted models on test data that the participants never see. When Evaluation Misleads shows what goes wrong when test data shapes the model, from tuning against a fixed benchmark to test items that were part of the training data of a language model.

Most models have **hyperparameters**: settings that the training algorithm does not learn but that we choose before training. The degree of the polynomial in Over- and Underfitting is one. Others are the number of layers of a ResNet, the activation functions, the optimization algorithm, the strength of the regularization, and the number representation, such as int8, fp16, or fp32. Choosing them adds an outer optimization loop around training. Again, the aim is not the lowest training loss but the best generalization.

Choosing the hyperparameters with the test set would leak information about the test data into the model, and the final test score would no longer estimate the performance on new data. The same holds for inspecting test errors to diagnose a model, because each fix inspired by the test set fits the model to it. Instead, we split the training data again, about 80% for training and 20% for a **validation set**. We train with each candidate set of hyperparameters on the training part, compare the candidates on the validation set, and use the test set only once, for the final model. Scores, Thresholds, and ROC Curves applies the same rule to the choice of a decision threshold.

When examples are not independent, a split must keep related examples together. Overlapping segments of one audio recording are near duplicates, so Classifying Audio holds out whole recordings instead of random segments.

k-Fold Cross-Validation

A single validation split has two drawbacks when data is scarce. The examples in the validation set are lost for training, and the validation score depends on which examples happened to land in it. **k-fold cross-validation** removes both. After the test set is set aside, the training data is divided into k parts of equal size, the **folds**, and each fold takes a turn as the validation set. A model must not be validated on examples it was trained on, because its score would then reward memorization. Each fold therefore needs its own model, trained on the other $k - 1$ folds, which makes k models in total. All k models use the same candidate set of hyperparameters; they differ only in the folds they were trained on. Every example validates one of these models and helps train all the others.

The average of the k validation scores rates this candidate. Since it includes a validation error for every training example, it no longer depends on one particular split. We repeat the procedure for every candidate set of hyperparameters and choose the candidate with the best average. Figure 0.9 shows one such cross-validation with five folds, with the validation fold marked in each of the five runs. The k models serve only

to rate their candidate and are discarded afterwards. With the chosen hyperparameters, we train the final model on all training data and assess it on the test set.



Figure 0.9: Five-fold cross-validation: each row is one training run, and the box marks its validation fold.

Typical values of k lie between 5 and 10. A larger k leaves more data for each training run at the price of more runs. With $k = N$, each fold holds a single example, which is called leave-one-out cross-validation. Kohavi compared the options in a large experiment and recommends stratified ten-fold cross-validation for model selection (see Further Reading). **Stratified** folds keep the class proportions of the whole training set in each fold. This matters for imbalanced data, where a rare class could otherwise be missing from some folds. Cross-validation costs k trainings for each set of hyperparameters. This is affordable for small models and often too expensive for large networks, which usually rely on a single validation split.

Cross-validation does not prevent overfitting; it measures it. It gives a more reliable estimate of the validation error, so that the choice of hyperparameters depends less on chance.

Example: Choosing the polynomial degree

We take one training set of ten points from the polynomial example of Over- and Underfitting and compare the degrees 0, 1, 3, 5, and 7 with five-fold cross-validation. Each fold holds two points, so each model is trained on eight points, too few for degree 9. The mean validation errors are:

Degree d	0	1	3	5	7
Cross-validation MSE	0.469	0.312	0.046	0.558	10.156

Degree 3 wins, and we train the final cubic polynomial on all ten points. The high errors of degrees 5 and 7 come from the folds that hold out the point at $x = 0$ or $x = 1$. There, the polynomial must extrapolate beyond its eight training points, and polynomials of high degree swing far away from the data. Repeated on 2,000 training sets drawn in the same way, each with a new random assignment of points to folds, cross-validation chooses degree 3 in 86% of the cases and degree 5 in 11%. Comparing the five degrees costs 25 trainings.

The Training Pipeline

Figure 0.10 puts the steps together. First, we split the data into a training set and a test set, keeping about 20% for the final assessment and neither using nor looking at it during training (step 1). Second, we split the training data into a training set and a validation set, again with about 20% for validation, or we use k-fold cross-validation when data is limited (step 2). Third, we train the model on the training set only (step 3). Training iterates over **epochs**, where one epoch is one pass over all training examples, until the performance converges or it becomes clear that the current hyperparameters are poor.

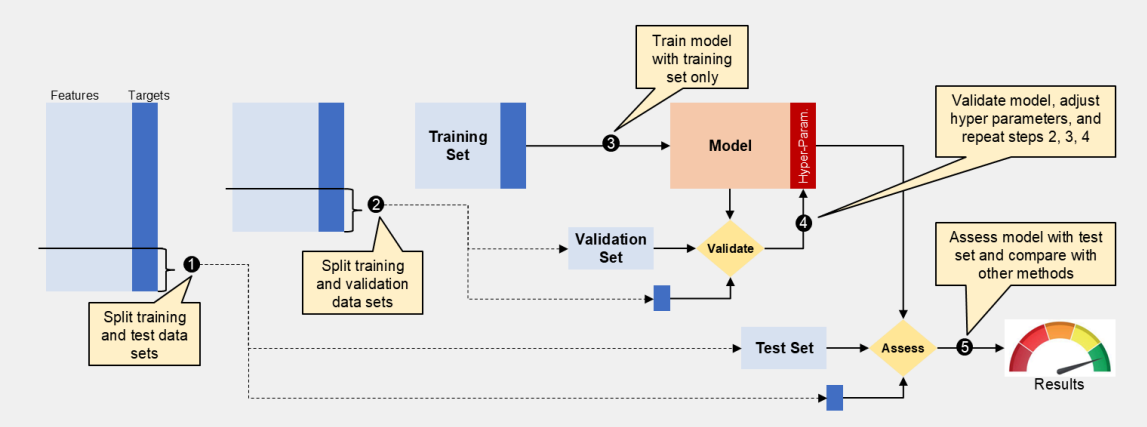


Figure 0.10: The model development pipeline in five steps, from splitting the data to the final assessment on the test set.

Fourth, we validate the model, adjust the hyperparameters, and repeat steps 2 to 4. With the best hyperparameters, we retrain the model, with more epochs if needed (step 4). Finally, we assess the model on the test set and compare it with other methods (step 5). If we iterate the whole process, each new round needs fresh test data, because the previous test set has influenced our decisions. In competitions, the contestants receive only the final assessment and cannot inspect where and why their model failed.

Summary

Key Takeaways

1. A program learns when its performance P at tasks T improves with experience E (Mitchell). Tasks include classification, regression, clustering, density estimation, anomaly detection, generation, transcription, and dimensionality reduction.
2. The loss used for training and the measure used to compare systems can differ. Regression trains with the mean squared error. Classification trains with the cross-entropy of soft assignments, because accuracy does not change smoothly with the parameters.
3. The softmax turns logits into probabilities. With a one-hot target, the cross-entropy loss is the negative log probability of the correct class, and its gradient with respect to the logits is $\hat{\boldsymbol{y}} - \boldsymbol{y}$.
4. Experience ranges from full supervision through semi-supervised, unsupervised, and reinforcement learning to self-supervised learning, which takes the targets from the data itself and trains the embedding models of this book.
5. Underfitting means high bias, overfitting means high variance. The expected squared error is the squared bias plus the variance plus the noise, and the best capacity minimizes the sum. In the polynomial example with ten points, this is degree 3.
6. The test set is used once, at the end. Hyperparameters are chosen on a validation set or by k-fold cross-validation, which estimates generalization more reliably but does not prevent overfitting.

Key Formulas

Key Formula: Mean Squared Error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2 \tag{0.13}$$

The average squared distance between prediction and target; large errors count far more than small ones.

Key Formula: Softmax

$$\hat{y}_k = \frac{\exp(o_k)}{\sum_{j=1}^{K} \exp(o_j)} \tag{0.14}$$

Turns logits into positive probabilities that sum to 1 and keep the order of the logits.

Key Formula: Cross-Entropy Loss and Its Gradient

$$J(\boldsymbol{y}, \hat{\boldsymbol{y}}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k, \qquad \frac{\partial J}{\partial o_k} = \hat{y}_k - y_k \tag{0.15}$$

The negative log probability of the correct class; its gradient with respect to each logit is the prediction minus the target.

Further Reading

- Mitchell, T. M. (1997). **Machine learning**. McGraw-Hill. Book page. The source of the definition of learning by task, performance measure, and experience that frames this chapter.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). **Deep learning**. MIT Press. Chapter 5, "Machine Learning Basics". The catalogue of tasks that A.1 follows, together with capacity, overfitting, and cross-validation; the whole book is free online.
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). **Dive into deep learning**. Cambridge University Press. Section "Softmax Regression". Derives the softmax and the cross-entropy loss with runnable code; the book is free online.

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). **The elements of statistical learning: Data mining, inference, and prediction** (2nd ed.). Springer. Book page with free PDF. Chapter 7 treats the bias-variance decomposition and cross-validation in depth.
- Kohavi, R. (1995). **A study of cross-validation and bootstrap for accuracy estimation and model selection**. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. PDF. A large experiment that compares numbers of folds and recommends stratified ten-fold cross-validation for model selection.
- Belkin, M., Hsu, D., Ma, S., & Mandal, S. (2019). **Reconciling modern machine-learning practice and the classical bias-variance trade-off**. *Proceedings of the National Academy of Sciences*, 116(32), 15849-15854. arXiv. Shows the double descent of the test error beyond the point where a model fits its training data exactly.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). **Generative adversarial nets**. *Advances in Neural Information Processing Systems*, 27. arXiv. Introduces the generator and the discriminator that train each other.
- Ouyang, L., et al. (2022). **Training language models to follow instructions with human feedback**. *Advances in Neural Information Processing Systems*, 35. arXiv. Describes reinforcement learning from human feedback, with a reward model learned from human rankings of answers.
- Sutton, R. S., & Barto, A. G. (2018). **Reinforcement learning: An introduction** (2nd ed.). MIT Press. Free online edition. The standard textbook on agents, rewards, and the balance of exploration and exploitation.
- Gupta, S. C. (2022). **MLOps: Machine learning life cycle**. ML4Devs. Article. Explains how the data, model, development, and operations cycles combine into one MLOps lifecycle.